

Final Ödev 1: Turing Makinesi ile Binary Çarpma Hesaplayıcı

Proje Adı

Tek Bantlı Turing Makinesi ile İkili Sayı Çarpma Makinesi

Amaç

Bu ödevin amacı, öğrencilerin Turing Makinesi ile binary sayı sistemi üzerinde çarpma işlemini gerçekleştirmelerini sağlamaktır.

Bu süreçte öğrenciler özellikle:

- Bant üzerinde birden fazla veriyi ayırt etmeyi
- Ayraç (delimiter) kullanarak veri parçalamayı
- Durum tabanlı işlem tasarlamayı
- Karmaşık bir işlemi adım adım modellemeyi

öğrenecektir.

Gerçek Dünya Bağlamı

Bilgisayarlar tüm aritmetik işlemleri binary sistemde gerçekleştirir. Çarpma işlemi işlemcilerde doğrudan yapılmaz; genellikle **kaydır ve topla (shift & add)** mantığı ile gerçekleştirilir.

Bu projede öğrenciler, bu süreci Turing Makinesi seviyesinde modelleyecektir.

Problem Tanımı

Python programlama dili kullanılarak bir **Turing Makinesi simülatörü** geliştirilecektir.

Bu makine, binary tabanda verilen iki sayıyı çarpacaktır.

Girdi Formatı

Kullanıcıdan iki adet binary sayı alınacaktır:

Birinci sayı: 11

İkinci sayı: 10

Program bu girişleri aşağıdaki bant formatına dönüştürmelidir:

11*10=

Kritik Tasarım Gereksinimi (Zorunlu)

Bu ödevin en önemli kısmı **operandların bant üzerinde doğru ayrıştırılmasıdır.**

- * karakteri iki sayıyı ayırmak için kullanılacaktır
- = karakteri sonuç alanını belirtir

Makine şu ayrımı açık şekilde yapmalıdır:

- *'ın sol tarafı → **Birinci sayı (multiplicand)**
- *'ın sağ tarafı → **İkinci sayı (multiplier)**

Makine:

1. Bant üzerinde * karakterini bulmalı
2. Sol tarafı birinci sayı olarak işlemeli
3. Sağ tarafı ikinci sayı olarak işlemeli
4. İşlem boyunca bu ayrımı korumalıdır

Bu ayırım yapılmadan çözüm **geçersiz sayılacaktır.**

Beklenen Çıktı

Sonuç: 110

Çünkü:

11 = 3

10 = 2

3 × 2 = 6

110

Çalışma Mantığı

Makine çarpma işlemini **kaydır ve topla (shift & add)** yöntemi ile gerçekleştirmelidir.

Adımlar:

1. Bantta * bulunur ve sayılar ayrılır
2. İkinci sayının en sağ bitinden başlanır
3. Her bit için:
 - Bit = 1 → birinci sayı kopyalanır ve sola kaydırılarak eklenir
 - Bit = 0 → sadece kaydırma yapılır
4. Sonuç = karakterinden sonra yazılır

Örnek İşlem

```
  11
× 10
-----
  00
+ 110
-----
 110
```

Adım Adım Çalışma (Örnek)

Başlangıç: 11*10=

- Adım 1: '*' bulundu → operandlar ayrıldı
Adım 2: sağdan ilk bit = 0 → işlem yapılmadı
Adım 3: ikinci bit = 1 → 11 sola kaydırıldı → 110
Adım 4: sonuç yazıldı

Sonuç: 110

Python Programı Şunları Yapmalıdır

Program:

1. Kullanıcıdan iki binary sayı almalıdır
2. Girdilerin yalnızca 0 ve 1 içerdiğini doğrulamalıdır
3. Girdiyi Turing bant formatına dönüştürmelidir (* ve = ile)
4. * karakterini kullanarak operandları ayırmalıdır
5. Turing Makinesi simülasyonunu çalıştırmalıdır
6. Her adımda:
 - mevcut durum
 - okunan sembol

- yazılan sembol
 - kafa hareketi
 - bant içeriği
gösterilmelidir
7. Sonucu:
- binary olarak
 - decimal karşılığıyla birlikte
göstermelidir

Sistem Gereksinimleri

1. Turing Makinesi bant yapısı
2. Okuma/yazma kafası
3. Durum kümesi
4. Giriş alfabesi (0,1,*,=)
5. Bant alfabesi
6. Geçiş fonksiyonu
7. Başlangıç durumu
8. Kabul durumu
9. Red durumu
10. Operand ayrıştırma mekanizması (zorunlu)
11. Adım adım simülasyon çıktısı

Teslim Edilecekler

1. Python kaynak kodu
2. Geçiş tablosu
3. Durum geçiş diyagramı
4. En az 5 farklı test örneği
5. Kısa proje raporu:
 - Problem tanımı
 - Turing Makinesi modeli
 - Binary sayı sistemi
 - Operand ayrıştırma yaklaşımı (özellikle açıklanmalı)
 - Durumların açıklaması
 - Geçiş mantığı
 - Girdi-çıkı örnekleri
 - Program ekran görüntüleri
 - Sonuç ve değerlendirme